

Práctica 4: Raft 2ª parte



**Escuela de
Ingeniería y Arquitectura**
Universidad Zaragoza

Rubén Agustín Galdeano (820829)
Antonio González Almela (143045)

Contenido

1.- Introducción	3
En esta fase final del desarrollo del servicio replicado basado en Raft, se completa el comportamiento iniciado en la Práctica 3 incorporando:	3
2.- Arquitectura General: Raft con Máquina de Estados	3
3.- Implementación del algoritmo Raft	3
3.1.- Estado persistente	3
3.2.- Estado volátil	4
3.3.- Estado volátil extra en líderes	4
3.4 Otros componentes clave del código	4
4.- Funcionamiento interno de Raft según la implementación	4
4.1.- Elecciones y liderazgo	4
4.2.- Replicación del log	4
4.3.- Compromiso por mayoría	4
4.4.- Replicación distribuida integrada	5
4.5.- Recuperación de seguidores atrasados	5
4.6.- Manejo de concurrencia	5
5.- Pruebas realizadas y análisis detallado de resultados	5
5.1.- Test T1 – Arranque inicial sin elección prematura	6
5.2.- Test T2 – Elección del primer líder	6
5.3.- Test T3 – Fallo del líder y elección de uno nuevo	7
5.4.- Test T4 – Compromiso de tres operaciones en clúster estable	7
5.5.- Test T5.1 – Acuerdo a pesar de la desconexión de un seguidor	8
5.6.- Test T5.2 – Sin acuerdo cuando se pierde la mayoría	8
5.7.- Test T5.3 – Concurrencia real al someter cinco operaciones	9

1.- Introducción

En esta fase final del desarrollo del servicio replicado basado en Raft, se completa el comportamiento iniciado en la Práctica 3 incorporando:

Compromiso correcto de entradas por mayoría.

- Aplicación ordenada a una máquina de estados KV.
- Gestión completa de logs inconsistentes mediante `nextIndex` y `matchIndex`.
- Recuperación tras fallos y reconexión de nodos.
- Comportamiento seguro bajo concurrencia real.
- Exposición de RPCs necesarias para pruebas de integración distribuidas.

La implementación se valida mediante el archivo `'testintegracionraft1_test.go'`, que levanta réplicas reales por SSH y simula fallos, desconexiones, actualización de logs y operaciones concurrentes.

El resultado final es un servicio replicado de tipo clave–valor (KV) fuertemente consistente, tolerante a fallos y correcto frente a escenarios reales de concurrencia y errores.

Esta memoria describe los cambios introducidos en `'raft.go'`, la arquitectura resultante y un análisis exhaustivo de los tests utilizados para comprobar la corrección del algoritmo Raft.

2.- Arquitectura General: Raft con Máquina de Estados

Raft por sí mismo no ejecuta operaciones del cliente. Raft solo replica y ordena comandos. La ejecución real ocurre en la máquina de estados, que es el servicio final ofrecido a los clientes.

En esta práctica:

- Raft garantiza el orden total de operaciones.
- El KV aplica los comandos exactamente una vez y en el mismo orden en todas las réplicas.
- La comunicación entre Raft y KV se realiza mediante el canal `canalAplicarOperacion`.

Así se logra un sistema replicado determinista, coherente y tolerante a fallos, que es la finalidad última del algoritmo Raft.

3.- Implementación del algoritmo Raft.

El fichero `raft.go` contiene una implementación completa del algoritmo Raft con los elementos siguientes:

3.1.- Estado persistente

- `currentTerm`
- `votedFor`
- `log`: lista de entradas de la forma (Term, Command)

3.2.- Estado volátil

- `commitIndex`: última entrada comprometida
- `lastApplied`: última entrada aplicada a la máquina de estados

3.3.- Estado volátil extra en líderes

- `nextIndex[]`: próxima entrada que debe enviarse a cada seguidor
- `matchIndex[]`: última entrada replicada correctamente en cada seguidor

3.4 Otros componentes clave del código

- Mutex global `mux` que protege todos los accesos al estado.
- Temporizador de elección con una primera ventana larga (2.5 segundos), necesaria para superar el Test T1.
- Temporizador aleatorio en rangos 150–300 ms tras la primera ejecución.
- Canal `canalAplicarOperacion` para enviar entradas comprometidas al servicio.
- Ciclo principal `run()` ejecutado en gorutina.
- Funciones para convertirse en follower, candidato y líder.
- Gestión completa del log y resolución de conflictos en `AppendEntries`.
- Protección frente a términos obsoletos y actualización inmediata al recibir términos mayores.
- Implementación de las RPC: `PedirVoto`, `AppendEntries`, `SometerOperacionRaft`, `ParaNodo`, `ObtenerEstadoNodo`, `ObtenerEstadoParaTest`, `ActivarTimers`.

Esta estructura cubre todos los requisitos del enunciado de la práctica 4.

4.- Funcionamiento interno de Raft según la implementación

Los puntos más relevantes de `raft.go` que permiten aprobar todos los tests son:

4.1.- Elecciones y liderazgo

- `startElectionLocked` inicia una elección.
- `PedirVoto` aplica correctamente la regla de log up-to-date.
- Un candidato obtiene votos y pasa a líder inicializando `nextIndex[]` y `matchIndex[]`

4.2.- Replicación del log

- `AppendEntries` verifica `prevLogIndex` y `prevLogTerm`.
- Si hay conflicto, se trunca el log en el seguidor.
- Se añaden las nuevas entradas.
- Se actualizan `nextIndex` y `matchIndex`.

4.3.- Compromiso por mayoría

`tryAdvanceCommitIndexLocked` realiza:

- Cómputo de cuántos nodos tienen `matchIndex >= N`.

- Compromete solo si hay mayoría.
- Solo compromete entradas si pertenecen al mandato actual (regla de seguridad).

Después:

- `applyCommittedEntriesLocked` avanza `lastApplied` y envía las operaciones a KV.

4.4.- Replicación distribuida integrada

La aplicación real de comandos (escribir o leer claves) NO se hace dentro de Raft. Se envía a `canalAplicarOperacion`, y la máquina de estados mantiene un servicio diccionario distribuido y replicado.

4.5.- Recuperación de seguidores atrasados

- `nextIndex` decrece dinámicamente si un seguidor rechaza entradas.
- Se reenvía todo el log hasta que coincide `prevLogIndex/prevLogTerm`.
- El seguidor se pone al día automáticamente.

4.6.- Manejo de concurrencia

Los accesos a:

- `log`
- `currentTerm`
- `commitIndex`
- `nextIndex`

Están protegidos por un mutex global, garantizando ausencia de condiciones de carrera.

5.- Pruebas realizadas y análisis detallado de resultados

La Práctica 4 se centra en validar exhaustivamente la implementación de Raft mediante un conjunto de pruebas de integración distribuidas incluidas en el archivo `testintegracionraft1_test.go`. Estas pruebas se ejecutan sobre tres máquinas físicas, arrancando procesos reales mediante SSH, simulando desconexiones, reelecciones, concurrencia y replicación bajo fallos.

Es importante destacar que las pruebas no son unitarias, sino que ejercitan todo el sistema en condiciones reales. Superar estos tests significa que la implementación cumple las garantías fundamentales del algoritmo Raft: seguridad, disponibilidad con mayoría, corrección frente a fallos y consistencia del log.

Para la implementación de los test hemos incorporado dos nuevos métodos RPC en cada nodo raft. El primero permite que cuando los nodos arrancan no comiencen, tras un tiempo aleatorio una elección de líder. Expresamente el primer test `'soloArranqueYparadaTest1()'` requiere que los nodos estén en funcionamiento pero no debe hacer ningún líder. El segundo permite inspeccionar el estado interno de un nodo, de esta forma podemos comprobar que tras someter diversas operaciones los valores de `'commitIndex'`, `'lastApplied'` y tamaño del log de operaciones tienen el mismo valor en todos los nodos. Estos nuevos métodos RPC son:

- ActivarTimers()
- ObtenerEstadoParaTest()

A continuación se analizan cada uno de los tests en profundidad, detallando el objetivo conceptual del test, las propiedades del algoritmo Raft que valida, la interacción exacta con el código raft.go y los resultados esperados y obtenidos. Esta sección constituye el núcleo técnico de la práctica.

5.1.- Test T1 – Arranque inicial sin elección prematura

Objetivo del test:

Verificar que, justo tras arrancar los tres nodos, ninguno se convierta en líder antes de que el test inspeccione su estado.

Razón de este test.

Raft establece que un nodo recién iniciado debe permanecer en modo Follower hasta que expire su temporizador de elección. Si los temporizadores se dispararan demasiado pronto, el test detectaría un líder prematuro, con mandato distinto de cero, indicando un mal funcionamiento del arranque.

Propiedades Raft validadas

- Estado inicial del término (currentTerm = 0)
- No existe líder inicial
- No se emiten heartbeats prematuros
- No hay elección todavía

Interacción con raft.go

El código incluye un retraso inicial ampliado (aprox. 2.5 segundos) en resetElectionTimerLocked(). Este retardo asegura que los temporizadores no se disparen antes de que el test consulte los estados.

Resultado esperado

Los tres nodos devuelven:

- Term = 0
- EsLider = false
- IdLider = -1

El test demuestra que el arranque es limpio, determinista y coherente con Raft.

5.2.- Test T2 – Elección del primer líder

Objetivo del test: Comprobar que, al activar los temporizadores, exactamente un nodo inicia una elección y obtiene la mayoría, convirtiéndose en el único líder.

Razón de este test: Un fallo común es que varios nodos se crean candidatos simultáneamente, provocando empates continuos o múltiples líderes.

Propiedades Raft validadas

- Temporización aleatoria entre nodos
- Solicitud de votos correcta
- Regla de log actualizado (up-to-date)
- Un único líder por mandato

Relación con raft.go: El test depende de las funciones `startElectionLocked`, el RPC `PedirVoto` y las transiciones `becomeCandidate` y `becomeLeader`.

Resultado esperado: Un único nodo pasa a líder. Los demás permanecen como followers. El mandato será mayor que cero.

5.3.- Test T3 – Fallo del líder y elección de uno nuevo

Objetivo: Demostrar que el sistema detecta la caída del líder y elige un nuevo líder automáticamente.

Secuencia del test:

1. Identificar el líder actual.
2. Simular su parada enviando RPC `ParaNodo`.
3. Esperar a que los followers detecten ausencia de heartbeats.
4. Verificar que se inicia una nueva elección y aparece un nuevo líder.

Razón de este test: Raft garantiza disponibilidad siempre que exista mayoría activa.

Propiedades Raft validadas:

- Detección de fallos mediante timeout
- Reinicio autónomo de elecciones
- Correcta actualización de `currentTerm` y vuelta a follower cuando corresponde
- Garantía de un único líder

Interacción con raft.go: El líder detenido deja de enviar `AppendEntries`. Los seguidores detectan el fallo y disparan elecciones.

Resultado esperado: Surge un nuevo líder distinto al anterior.

5.4.- Test T4 – Compromiso de tres operaciones en clúster estable

Este es el primer test que estresa completamente la parte de consenso y compromiso del log.

Objetivo: Validar que en un entorno sin fallos:

- El líder replica operaciones en los seguidores.
- La mayoría reconoce esas entradas.
- El líder actualiza `commitIndex`.
- `lastApplied` avanza y las entradas se aplican a la máquina de estados KV.

Propiedades Raft validadas

- Replicación correcta
- Avance de `matchIndex` en seguidores

- Compromiso por mayoría
- Aplicación secuencial en todos los nodos
- Igualdad completa de estado entre nodos

En la Interacción con raft.go intervienen:

- SometerOperacionRaft
- AppendEntries
- tryAdvanceCommitIndexLocked
- applyCommittedEntriesLocked

Resultado esperado:

- commitIndex = 3
- lastApplied = 3
- logLength >= 4

Los nodos quedan completamente sincronizados.

5.5.- Test T5.1 – Acuerdo a pesar de la desconexión de un seguidor

Objetivo

Simular un fallo parcial (solo un seguidor cae) y comprobar que el sistema sigue progresando, al existir mayoría activa (2/3).

Propiedades Raft validadas:

- Disponibilidad con mayoría
- Compromiso continuo aun con fallos parciales
- Sincronización al reconectar el seguidor
- Uso correcto de nextIndex y matchIndex
- Reparación de logs desactualizados

Relación con raft.go: El líder replica solo al seguidor disponible. Al reconectar el nodo caído, pushLogToPeer actualiza su log.

Resultado esperado, tras reconectar el seguidor:

- commitIndex >= 4

5.6.- Test T5.2 – Sin acuerdo cuando se pierde la mayoría

Este test verifica la propiedad de seguridad más importante de Raft.

Objetivo: Asegurar que si solo queda 1 nodo activo de 3, el líder puede recibir operaciones pero no puede comprometerlas.

Razón de este test: Raft garantiza que ningún líder puede comprometer entradas sin mayoría.

Propiedades Raft validadas:

- El líder acepta operaciones pero solo las añade a su log local

- `commitIndex` no avanza sin mayoría
- No se vulnera la seguridad del sistema
- Al reconectar los nodos, todos reciben las entradas pendientes
- El avance del compromiso se produce solo cuando regresa la mayoría

Interacción con `raft.go`:

- `tryAdvanceCommitIndexLocked` no debe avanzar `commitIndex` mientras `matchIndex` no alcance mayoría.

Validación:

1. Se compromete `k1` con 3 nodos.
2. Se apagan 2 seguidores.
3. Se añaden entradas al log del líder.
4. `commitIndex` debe quedarse en 1.
5. Tras reconectar, los seguidores se ponen al día y se comprometen todas las operaciones.

Resultado esperado:

- Sin mayoría, `commitIndex` no avanza: `commitIndex` = 1.
- Tras reconectar, `commitIndex` $\geq$ 4.

5.7.- Test T5.3 – Concurrencia real al someter cinco operaciones

Objetivo: Evaluar el comportamiento bajo concurrencia real, sometiendo varias operaciones en paralelo.

Propiedades Raft validadas:

- Exclusión mutua correcta mediante mutex
- Log consistente bajo solicitudes concurrentes
- Replicación ordenada
- Ausencia de corrupción
- Mismo término en todos los nodos
- Progreso bajo carga

Relación con `raft.go`

Este test fuerza el uso concurrente de:

- `SometerOperacionRaft`
- `AppendEntries`
- `tryAdvanceCommitIndexLocked`
- Bloqueo mediante mux

Resultado esperado:

- `commitIndex` $\geq$ 6
- Todos los nodos tienen el mismo término
- `logLength` consistente en todos